REVIEW III · FINAL DEMONSTRATION AND INNOVATION

GraphRAG-Powered Cyber Threat Intelligence Platform

A multi-model NoSQL approach: Neo4j · MongoDB · Redis · Qdrant · local LLM

Shivansh Sharma
23BCE0254

Himali Sharma
23BCE0317

Aayush Sinha
23BCE0558

Guide: Dr. Lydia Jane G · VIT Vellore · October 2026

THE PROBLEM

Threat intelligence is four different shapes of data

Relationships

Which actors share C2 infrastructure with APT28?

→ graph

Reports

STIX bundles and analyst reports, each with different nested fields

→ document

Live indicators

Is 185.86.77.12 still on the blocklist? It must expire on time.

→ key–value + TTL

Narrative

What else looks like this spearphishing lure?

→ vector similarity

One engine fits at most one of these. And an LLM answering CTI questions must not invent attributions: every claim has to be traceable to stored evidence.

REVIEW III

A complete, measured system

4

NoSQL engines

graph, document, key–value, vector — one job each

36

API endpoints

JWT roles, TLP on every read, fail-closed enforcement

34

automated tests

18 unit + 16 integration, run in CI on every push

100%

gold evidence recalled

six acceptance questions, edges and reports

9.5 s

GraphRAG p95

target < 15 s (was 52.3 s)

200×

corpus scaling tested

15,012 nodes, 2,400 reports

185 ms

retrieval p95 at 200×

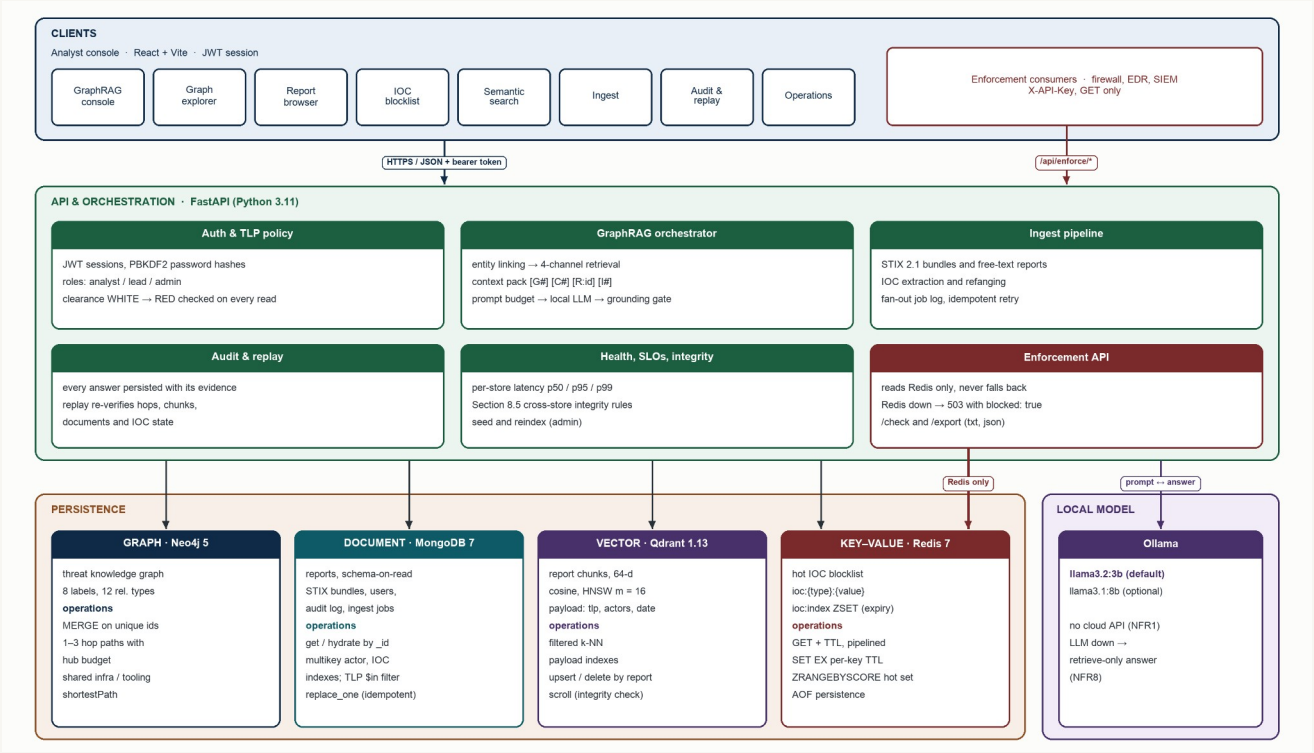
was 14.7 s before the fixes

4

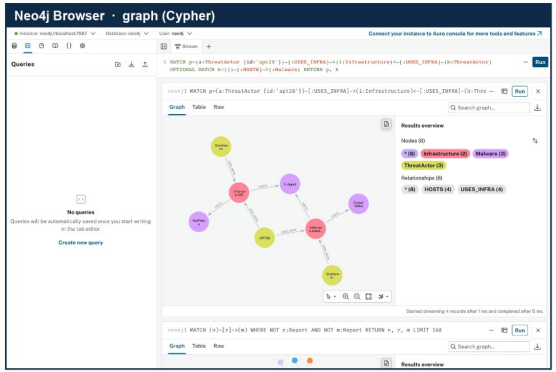
scalability defects fixed

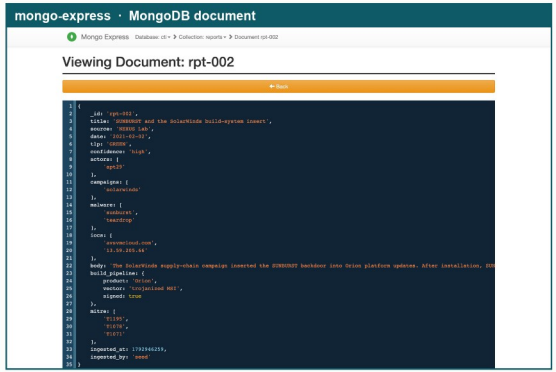
found by our own benchmark harness

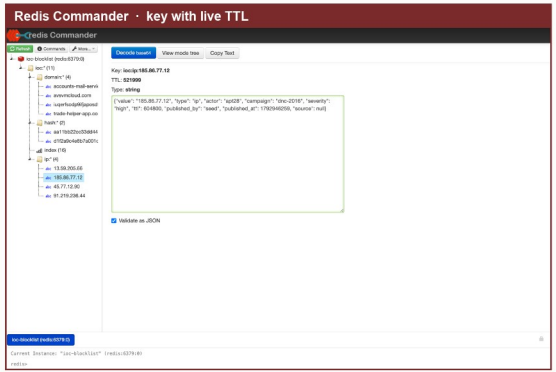
Each store does one job; the API owns policy

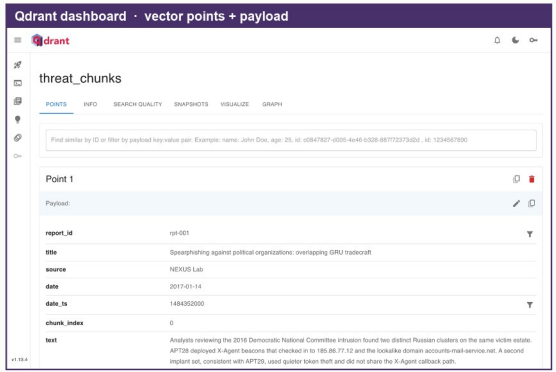


Four real database servers, each in its own console









Open them live

Neo4j :7474

MongoDB :8081

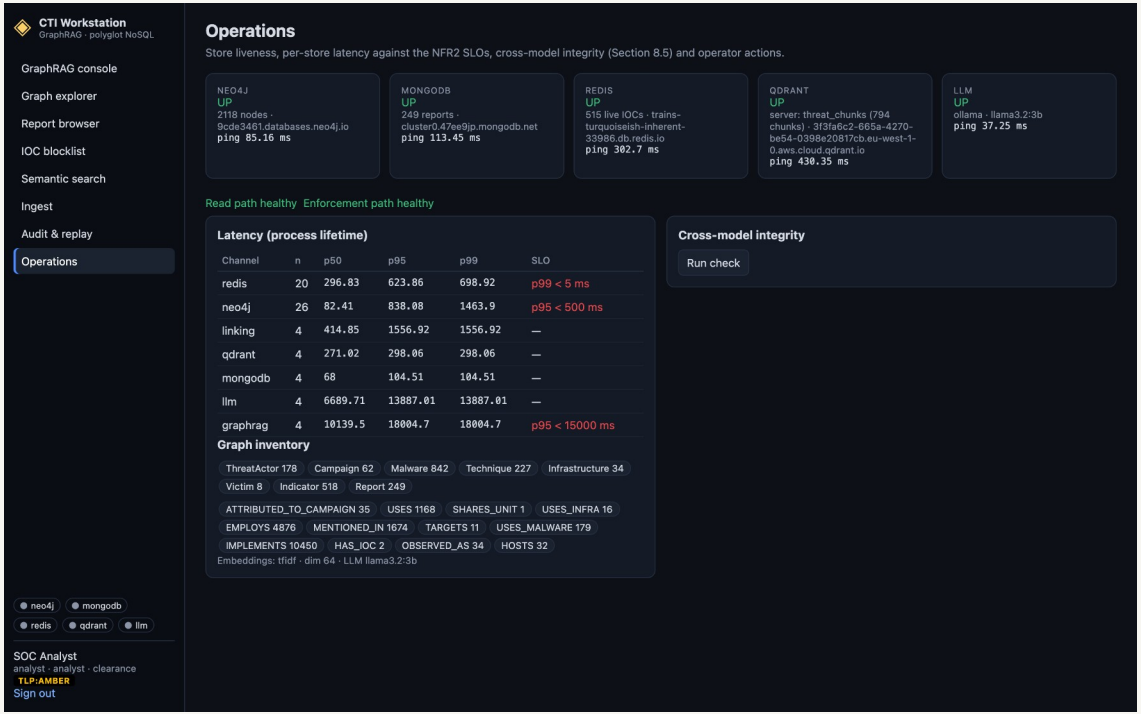
Redis :8082

Qdrant :6333/dashboard

make dbui starts the MongoDB and Redis consoles (read-only)

HOSTED · REAL DATA

The same platform on four cloud databases



Hosted on

AuraDB · Neo4j 5.27-aura

Atlas · MongoDB 8.0.34

Redis Cloud · Redis 8.6.2

Qdrant Cloud · Qdrant 1.20.0

Real feeds loaded

MITRE ATT&CK v20.0

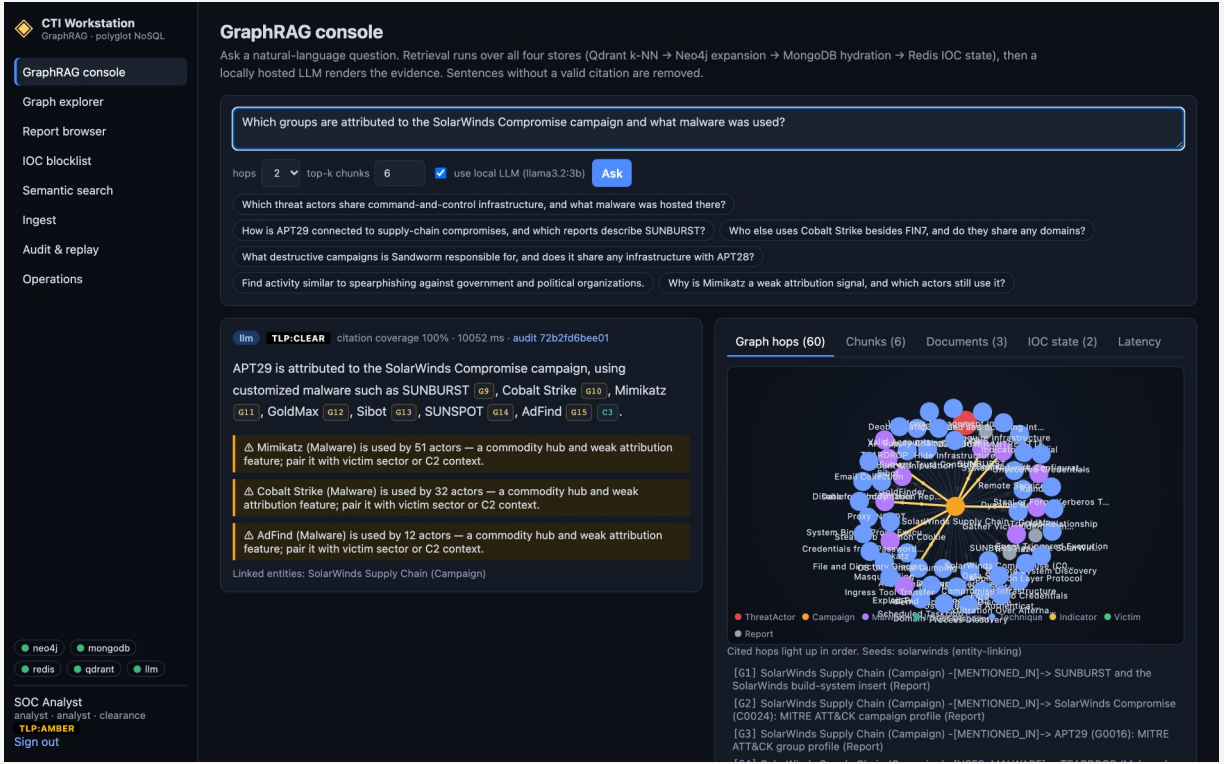
abuse.ch Feodo Tracker · Emerging Threats

2,118 nodes · 18,478 edges

249 reports · 794 chunks · 515 live IOCs

HOSTED · REAL DATA

GraphRAG answers from ATT&CK in the cloud



On the hosted stores

gold edges 100% · gold reports 100%

citation coverage 0.87

integrity: 249 reports in all stores

16/16 integration tests pass

latency includes the internet: GraphRAG p95 18.3 s

SECURITY · IP GATE

A blocklisted IP cannot use the platform

Requests blocked by the API (2)

Every API request from an IP that is live on this blocklist gets HTTP 403. Each refusal is logged in MongoDB (`blocked_requests`) and deleted automatically by a TTL index.

When	Source IP	Request	Blocked by	Log expires
19:24:22	50.16.16.211	POST /api/auth/login	ioc:ip:50.16.16.211 · critical	02/11/2026, 19:24:22
19:24:22	50.16.16.211	GET /api/reports	ioc:ip:50.16.16.211 · critical	02/11/2026, 19:24:22

The same key read straight from Redis Cloud (Redis Commander, read-only): value and TTL in seconds

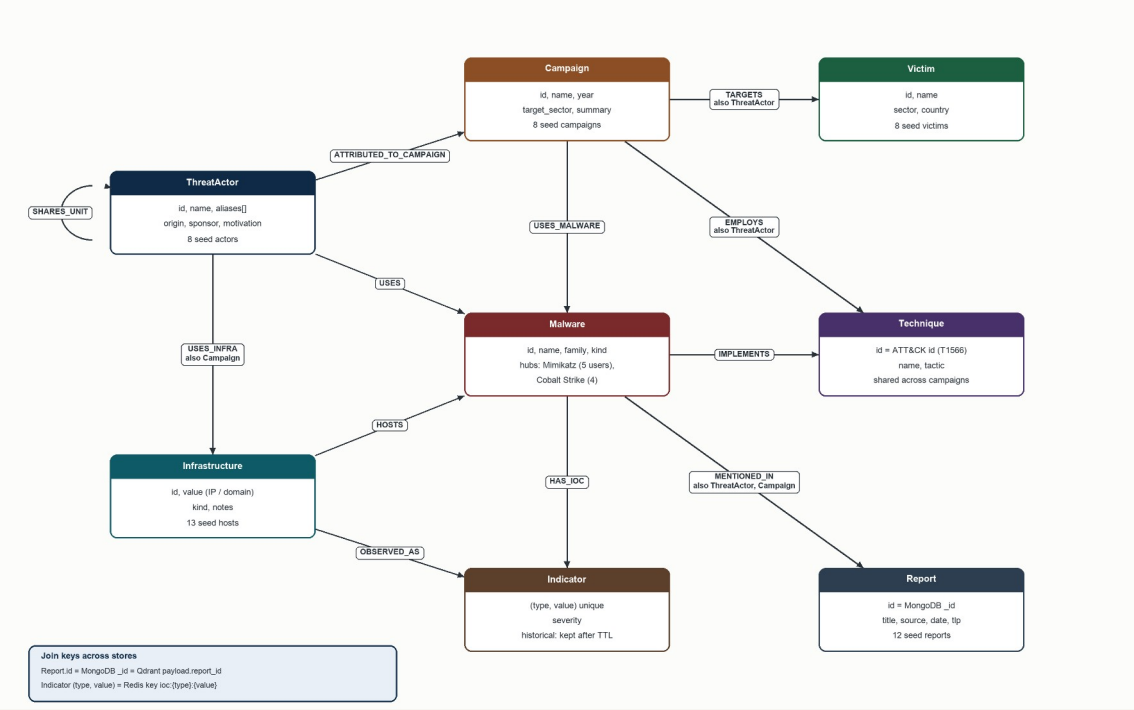
```
DBSIZE
516
ZCOUNT ioc:index -inf +inf
515
GET ioc:ip:50.16.16.211
"{\"value\": \"50.16.16.211\", \"type\": \"ip\", \"actor\": null, \"campaign\": null, \"severity\": \"critical\", \"ttl\": 604800, \"published_by\": \"import-feeds\", \"published_at\": 1793042268, \"source\": \"abuse.ch Feodo Tracker\", \"malware\": \"QakBot\"}"
TTL ioc:ip:50.16.16.211
603771
GET ioc:ip:34.204.119.63
"{\"value\": \"34.204.119.63\", \"type\": \"ip\", \"actor\": null, \"campaign\": null, \"severity\": \"high\", \"ttl\": 259200, \"published_by\": \"import-feeds\", \"published_at\": 1793042268, \"source\": \"abuse.ch Feodo Tracker\", \"malware\": \"QakBot\"}"
TTL ioc:ip:34.204.119.63
258166
```

Verified on the cloud

- 50.16.16.211 → HTTP 403
- health check → 200
- refusal log: TTL index expires_at_1 in Atlas
- expired log entry deleted in 36 s
- 45 s block → admitted after 47 s
- Redis down → gate fails open; /enforce fails closed

DATA MODEL

One graph, three join keys



Join keys

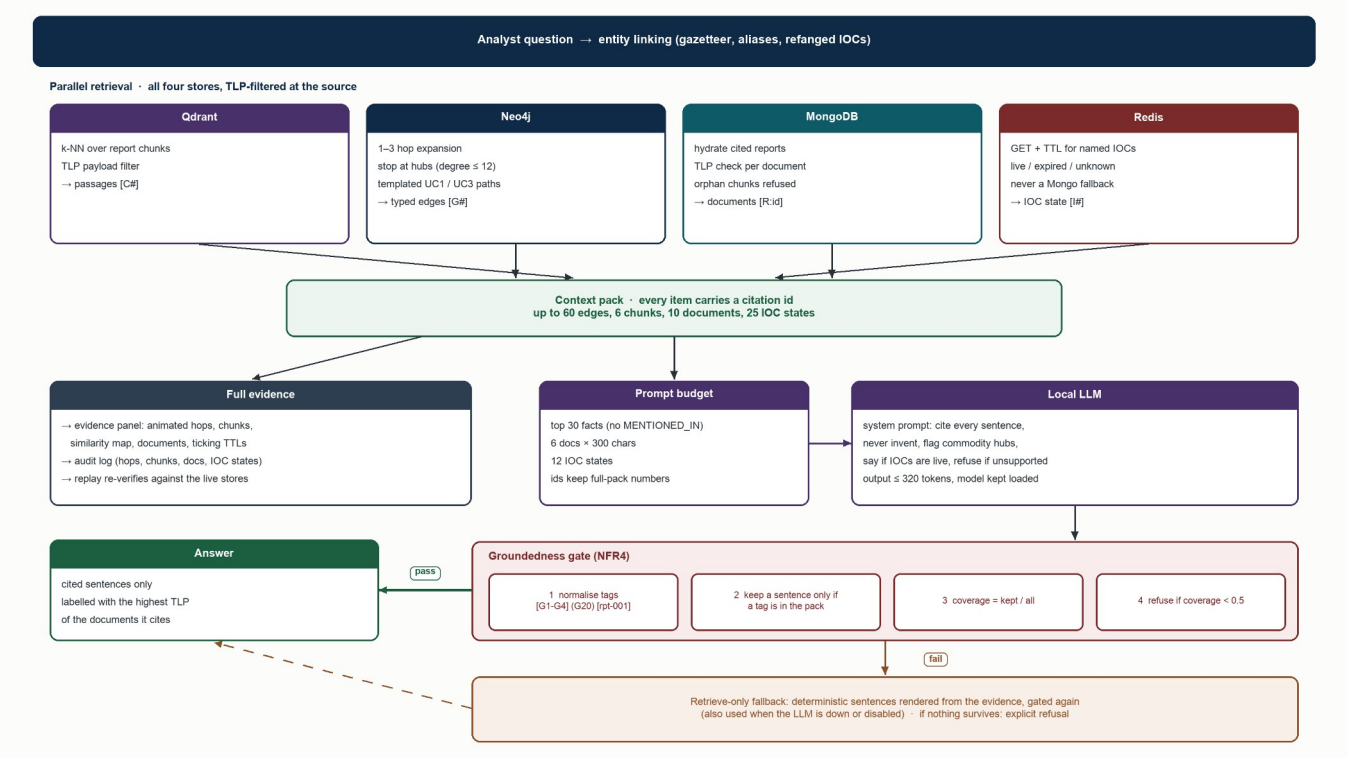
Report.id = MongoDB _id = Qdrant report_id

Indicator (type, value) = Redis key ioc:{type}:{value}

Indicators stay in the graph after their TTL: enforcement and history can disagree (UC4).

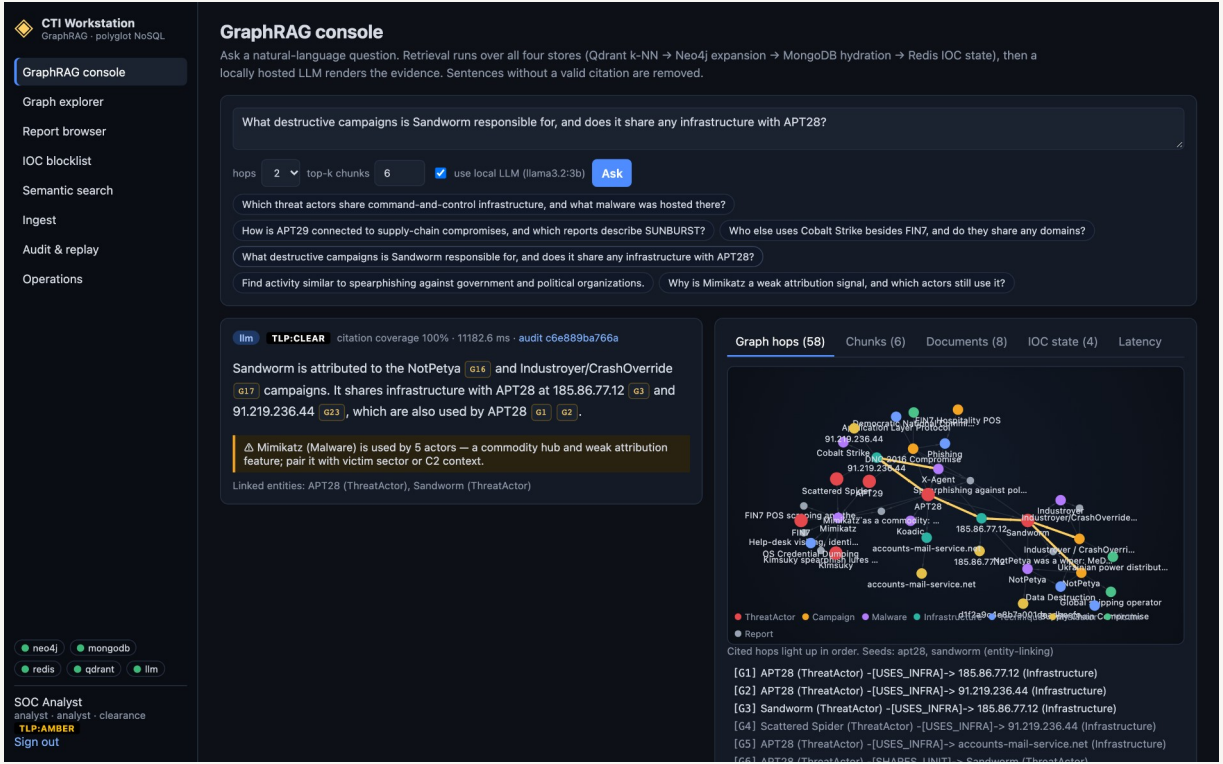
Integrity check: 3 hard + 2 informational cross-store rules.

Four-channel retrieval, a local LLM, and a gate



LIVE SYSTEM

Every sentence is cited and every hop is visible



Reading an answer

llm / retrieve-only mode

TLP label of the cited reports

citation coverage

G · C · R · I citation chips

⚠ commodity-tool caution

evidence: hops, chunks, documents, IOC state, latency

audit link → replay

INNOVATION

What is new in this design

Four-store GraphRAG

typed citations across graph, documents, vectors and cache

Groundedness gate

uncited sentences removed; refusal + evidence-only fallback

Hub-aware traversal

stops at commodity tools; weak-attribution caution

TLP inside retrieval

payload filter, $in filter, graph filter; answers labelled

Fail-closed enforcement

Redis-only API; 503 blocked:true when Redis is down

Replayable answers

re-verify every hop, chunk and IOC weeks later

Local AI, prompt budget

3B model meets the 15 s SLO with no cloud API

Self-measuring

evaluation gate + benchmark harness in the code base

Using each engine beyond CRUD

Neo4j

- variable-length paths with COUNT {} predicates (hub budget)
- weighted path ranking with reduce()
- composite (type, value) uniqueness
- UNION-of-labels index seeks
- CALL … IN TRANSACTIONS bulk writes

MongoDB

- schema-on-read report fields
- multikey indexes on actors and IOCs
- explain() to verify IXSCAN
- idempotent replace_one upserts
- collStats for index sizing

Redis

- per-key TTL (SET EX)
- sorted-set expiry index
- pipelined GET + TTL
- hot set via ZRANGEBYSCORE
- AOF persistence

Qdrant

- HNSW, cosine, m = 16
- keyword + integer payload indexes
- filtered k-NN
- tunable ef (recall vs speed)
- exact search as ground truth

Each claim is measured, not assumed: Neo4j PROFILE, MongoDB explain() and exact k-NN as ground truth back every index and traversal choice (Section 9).

VALIDATION

Tested at three levels, on every push

Test tiers

18 unit tests — TLP, parsing, linker equivalence, gate

16 integration tests — real API, live stores

Evaluation gate — 6 questions, exits non-zero on a miss

Benchmarks — 6 experiments

CI — all of the above, fresh containers, every push

Acceptance question	Edges	Reports	Coverage
Shared C2 + hosted malware	2/2	1/1	0.50
APT29 ↔ SUNBURST supply chain (gate fallback)	1/1	1/1	1.00
Cobalt Strike users besides FIN7	2/2	1/1	1.00
Sandworm campaigns + APT28 infra	2/2	1/1	1.00
Spearphishing analogue search	0/0	1/1	0.67
Why Mimikatz is weak attribution	2/2	1/1	0.60

All gold evidence recalled · no refusals · mean coverage 0.79

PERFORMANCE · SERVICE LEVELS

Every latency objective is met

Redis blocklist GET

1.08 ms

p99 · target < 5 ms

0.534 ms at 500,000 keys

Neo4j 1–3 hop

15.73 ms

3-hop p95 · target < 500 ms

1-hop 12.39 ms · 2-hop 35.52 ms

GraphRAG with LLM

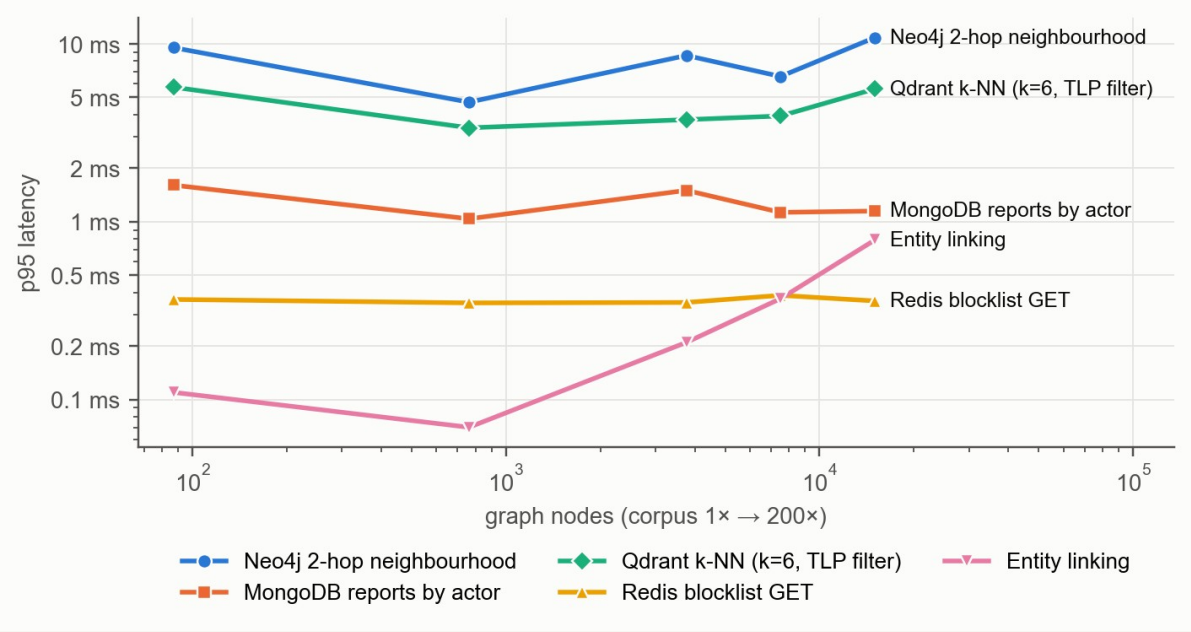
9.5 s

p95 · target < 15 s

llama3.2:3b · was 52.3 s

Measured with make eval and make bench · Apple M3 Pro host, Docker VM with 2 vCPU / 4 GB shared by all engines

1× → 200×: store queries stay flat

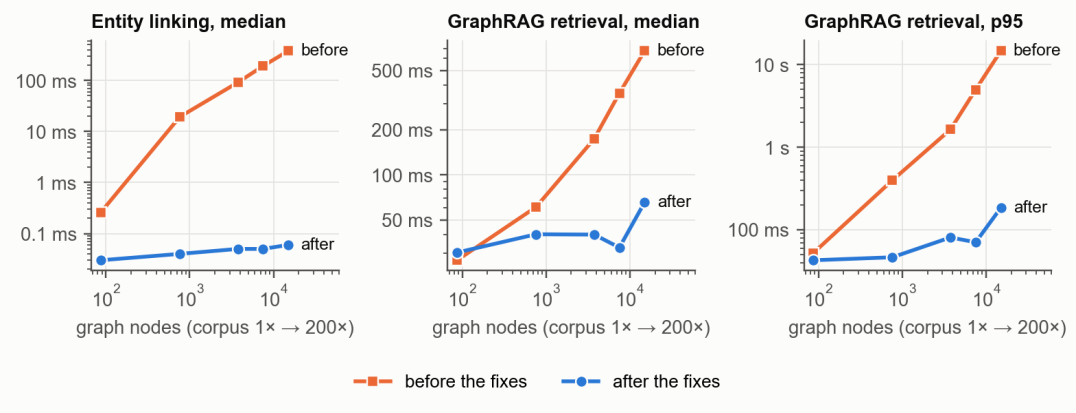


At 200× (15,012 nodes)

Neo4j 2-hop p95 10.8 ms
MongoDB by actor 1.15 ms
Qdrant k-NN 5.58 ms
Redis GET 0.36 ms
Entity linking 0.79 ms

Evidence capped at 60 edges by the hub budget

Four defects, invisible at seed scale



Linear entity linking → inverted index — **379 ms → 0.06 ms**

Unbounded query + N+1 full scans → LIMIT + batched index seeks — **p95 14.7 s → 185 ms**

Whole-graph join on ingest → plan-scoped join — **1.5 → 31.6 reports/s**

Shared vector client under load → one client per thread — **HTTP 500 → 0 errors**

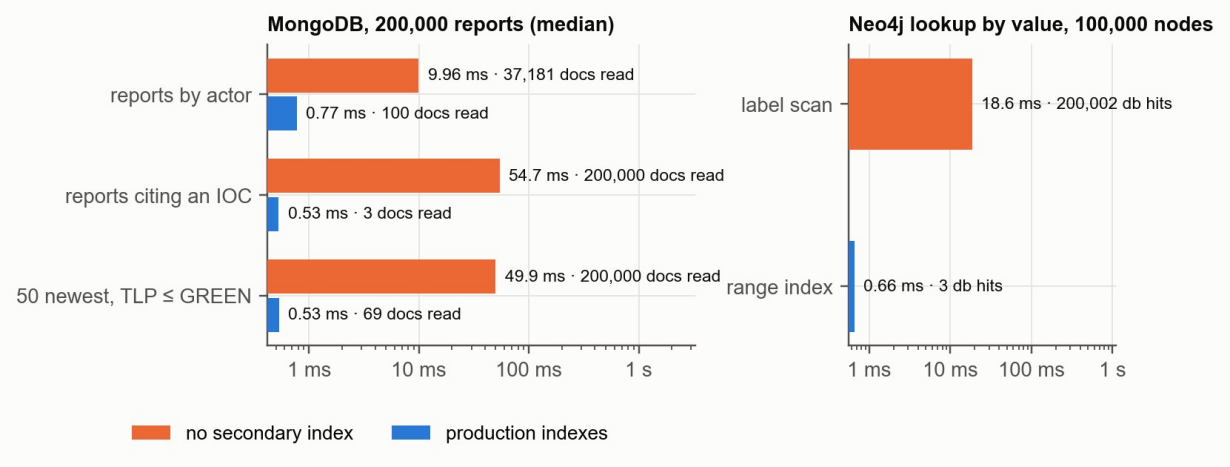
At 200×

retrieval median
675 ms → 65.4 ms

retrieval p95
14.7 s → 185 ms

ingest
1.5 → 31.6 reports/s

Indexes turn scans into seeks



103×

MongoDB IOC lookup

200,000 → 3 documents read

28×

Neo4j lookup by value

200,002 → 3 db hits

Plans verified with MongoDB explain() (COLLSCAN → IXSCAN) and Neo4j PROFILE (NodeByLabelScan → NodeIndexSeek).

O(1) enforcement, accurate vector search

Redis at 500,000 keys

0.29 ms

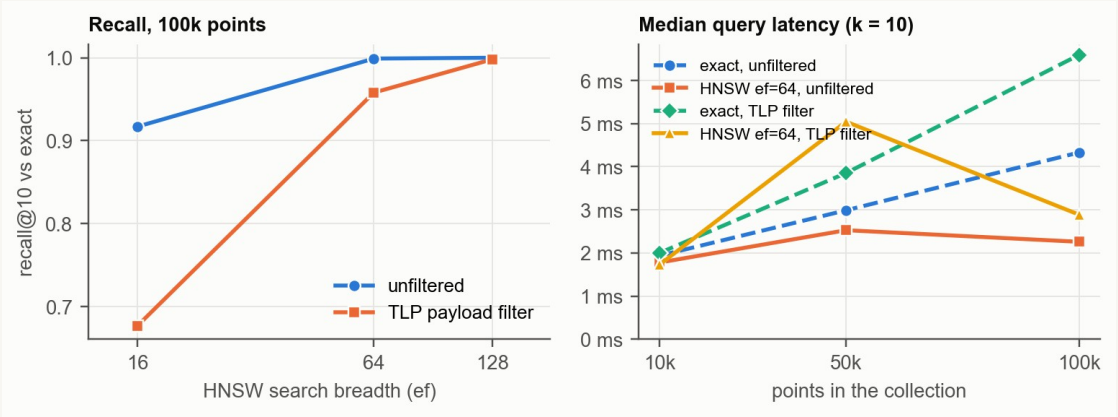
GET + TTL median

p99 0.53 ms · flat from 1k keys

1.0 s

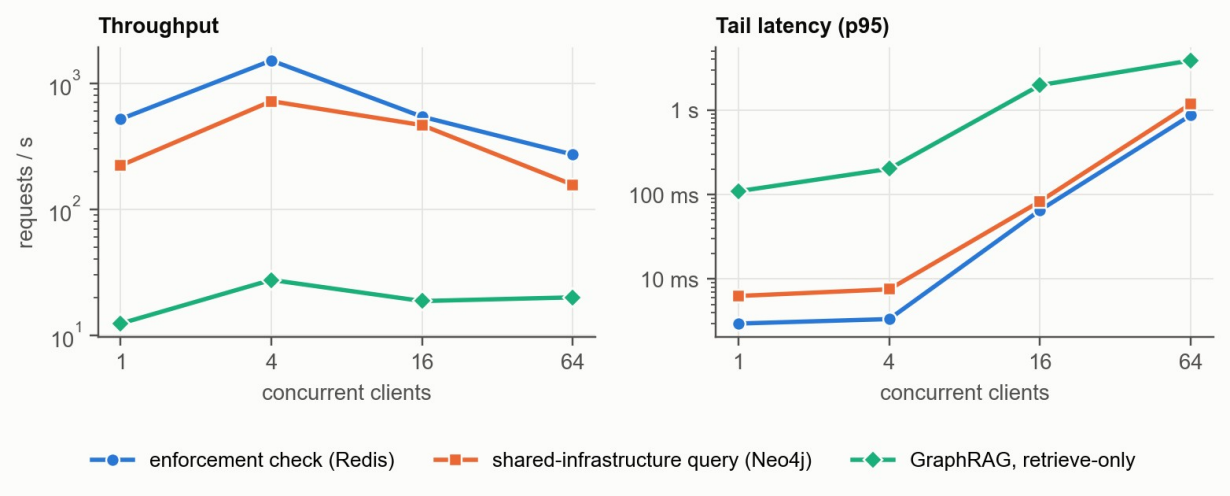
full SCAN (non-key lookup)

grows linearly · 191.6 MB memory



Qdrant, 100,000 points: HNSW 2.26 ms vs exact 4.33 ms · recall@10 0.999 (ef 64)

One worker: ~1,500 checks/s, then queueing



Takeaways

peak 1,512 enforcement checks/s

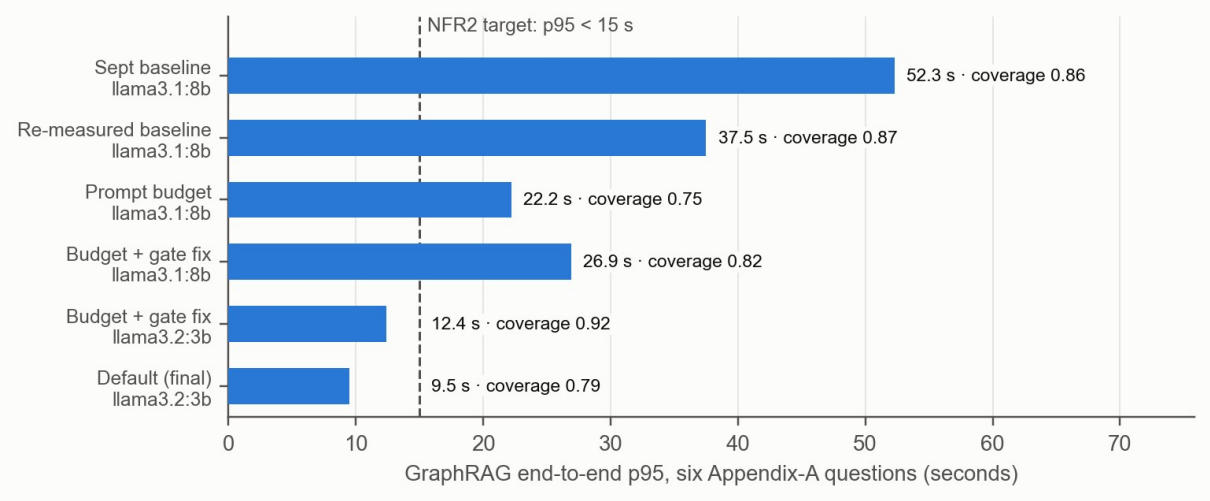
0 errors at 64 clients

knee at 4 clients: one Python worker on 2 vCPUs

scale out with workers or replicas — the stores are not the limit

PERFORMANCE · LLM LATENCY

From 52.3 s to 9.5 s without a GPU server



Why it got faster

prompt evaluation was 17 s of a 24.5 s call

prompt budget: ~½ the characters

output cap 320 tokens, model kept loaded

default llama3.2:3b; llama3.1 is one setting away

REQUIREMENTS

All 20 requirements met, all 5 gaps closed

✓ FR1 ingest	✓ FR2 multi-hop	✓ FR3 blocklist	✓ FR4 search	✓ FR5 GraphRAG
✓ FR6 hydration	✓ FR7 evidence UI	✓ FR8 corpus	✓ FR9 health/SLO	✓ FR10 replay
✓ FR11 TLP	✓ FR12 publish	✓ NFR1 local only	✓ NFR2 latency	✓ NFR3 isolation
✓ NFR4 grounded	✓ NFR5 reproducible	✓ NFR6 ethics	✓ NFR7 usability	✓ NFR8 availability

Review I gaps closed: G1 one engine for four workloads · G2 no graph + enforcement plane · G3 no TTL cache or vectors beside the graph · G4 under-constrained RAG · G5 un-auditable answers

LIVE DEMO

Six minutes, four use cases

UC1

Shared C2 with APT28

Sandworm via 185.86.77.12; cited rpt-001; IOC state

UC2

Similar spearphishing

Qdrant passages → APT28 / APT29 → reports

UC3

Mimikatz attribution

five users, commodity-hub caution

UC4

Indicator expiry

TTL hits zero; graph still remembers

TLP

Sign in as junior

AMBER reports vanish; 403 on rpt-001

Ops

Ingest, replay, health

STIX fan-out; replay re-verifies; SLO view

Before the demo: make seed 5 minutes ahead (fresh TTLs) · warm the model with one question · if the LLM stalls, untick “use local LLM” — answers stay cited

What we would do next

Limitations

- single-node stores; one API worker
- feeds are imported, not streamed; 200× scaling is synthetic
- free-tier cloud adds internet latency
- 3B model is terser than 8B
- TF-IDF embeddings by default
- lab secrets in .env.example

Next steps

- Uvicorn workers / replicas; Neo4j and Mongo clustering
- TAXII connector with scheduled refresh
- stream answers, gate on completion → 8B feels interactive
- nomic-embed-text as default embedder
- secrets from a vault; SSO for analysts

None of these needs a design change: each is configuration, scale-out or a new connector.

Each store does what it is best at — and every answer can be traced.

9.5 s
GraphRAG p95 (< 15 s)

100%
gold evidence recalled

185 ms
retrieval p95 at 200×

34
automated tests in CI

Thank you — questions?

github.com/Shivansharmaaaa/graphrag